Penetration Testing Report

---

1. Executive Summary

This report was written by Andrei S and it is intended to asses the vulnerabilities found in hackthissite.org's basic challenges. Completing these levels helped me understand how certain web vulnerabilities can be exploited and how they can be mitigated. Vulnerabilities found in these challenges include: left credential in client site code, locally manipulating html code leading to credential redirection, command injection, weak encryption mechanisms, directory traversal and storing of cookie information in plaintext.

---

2. Scope

Scope of this penetration test consists of basic web challenges 1-10. Every challenge was completed within hackthissite.org's browser environment and no other software tools were used to complete these challenges. By increasing the chance of exploiting vulnerabilities, open source intelligence was used during penetration testing.

---

3. Methodology

For the simpler challenges, client code analysis was sufficient in exploiting the vulnerabilities. Difficulty was linearly increasing, with some requiring input manipulation and locally changing client side code to steal credentials, using directory traversal to access certain unprotected resources and injecting shell commands into invalidated input.

---

4. Per-level Vulnerability Cases

Level 1

Vulnerability Category: Client-side credential exposure
Description: Password embedded directly in HTML source.
Impact: Immediate unauthorized access; zero barrier to entry.
Exploit Procedure: View page source, extract hardcoded password, submit it.
Evidence: Exposed password string visible in markup.
Risk Rating: High
Recommendation: Remove all authentication secrets from client-side code; enforce server-side credential verification.

Level 2

Vulnerability Category: Authentication logic flaw
Description: Empty password accepted as valid input.

Impact: Total authentication bypass.
Exploit Procedure: Submit an empty string; system grants access.
Evidence: Server returns success with null password field.
Risk Rating: High
Recommendation: Implement strict input validation and mandatory non-empty credential checks on the server.

Level 3

Vulnerability Category: Directory traversal and improper file permissions
Description: Password file exposed via direct path access.
Impact: Leakage of authentication secrets.
Exploit Procedure: Navigate to predictable file path to retrieve password.
Evidence: Retrieved plaintext password file.
Risk Rating: High
Recommendation: Enforce directory access restrictions; block traversal patterns; remove sensitive files from public directories.

Level 4

Vulnerability Category: Client-side trust and manipulable form actions
Description: HTML form action can be edited to redirect the submission to a different address.
Impact: Email redirection and workflow compromise.
Exploit Procedure: Modify form action in developer tools before submission.
Evidence: Email sent to attacker-controlled destination.
Risk Rating: Medium
Recommendation: Validate form submission targets on the server; disallow arbitrary destinations.

Level 5

Vulnerability Category: Client-side trust and manipulable form fields
Description: Same pattern as Level 4; email routing can be altered by modifying HTML.
Impact: Unauthorized alteration of business logic.
Exploit Procedure: Edit form attributes; submit manipulated request.
Evidence: Form processed with modified destination data.
Risk Rating: Medium
Recommendation: Enforce server-side verification of all routing parameters; eliminate reliance on client-supplied actions.

Level 6

Vulnerability Category: Weak cryptography

Description: Password protected by simple, reversible encoding.
Impact: Trivial decryption; false sense of security.
Exploit Procedure: Decode using common reversible transformation.
Evidence: Encoded string mapped directly to plaintext.
Risk Rating: Medium
Recommendation: Use modern cryptographic hashing (with salt), avoid reversible schemes entirely.

Level 7

Vulnerability Category: Command injection and exposed password file
Description: User-supplied input executed as Linux command; sensitive file accessible.
Impact: Arbitrary command execution; credential leakage.
Exploit Procedure: Inject commands through input field; read publicly reachable password file.
Evidence: Shell output returned in HTTP response.
Risk Rating: High
Recommendation: Sanitize all command-bound input; remove direct system call pathways; harden file permissions.

Level 8

Vulnerability Category: Server-side script execution (shtml)
Description: shtml files allow execution of embedded shell commands.
Impact: Directory enumeration and potential command execution.
Exploit Procedure: Craft shtml file invoking directory listing commands.
Evidence: Returned file structure output.
Risk Rating: High
Recommendation: Disable server-side includes or restrict allowed directives; isolate execution contexts.

Level 9

Vulnerability Category: Directory breakout
Description: Able to navigate outside web root.
Impact: Exposure of filesystem beyond intended boundaries.
Exploit Procedure: Traverse upward through directory structure.
Evidence: Access to parent directories not meant for public exposure.
Risk Rating: High
Recommendation: Enforce chroot-like directory confinement; normalize and validate paths server-side.

Level 10

Vulnerability Category: Insecure cookie handling

Description: Cookies stored in plaintext; modifiable without detection.

Impact: Privilege escalation or forced authentication states.

Exploit Procedure: Edit cookie values in browser; reload page to gain access.

Evidence: Application accepts modified cookie as valid.

Risk Rating: High

Recommendation: Sign or encrypt cookies; validate integrity server-side; avoid storing sensitive state client-side.

---

5. Conclusion

These vulnerabilities were exposed intentionally by the author for educative purposes. Tested levels expose fundamental failures: client-side trust, weak authentication, improper access control, insecure storage, and direct code or command execution paths. Each flaw illustrates how minimal validation and misconfigured environments lead to full compromise. Strengthening server-side controls and eliminating reliance on client-supplied data mitigates all demonstrated weaknesses.